Full codebook and definitions

	Code	Definition
1	**First move**	Codes related to the first set of changes made by a student, in relation to the first time they ran the code.
	Made changes before running code	Changes to the program are made before the code is run.
	Ran code before making changes	Ran the code before any changes at all were made to the program, meaning the first run was identical to the original state of the program.
2	**Positive debugging indicators**	Behaviour related to the resolution of errors that aids in the process of debugging
	Entered incorrect input	Where a student has inputted a string that has caused a correctly running program to crash, perhaps to test the program's robustness
	Made improvements to program	Changes that are made that do not improve the correctness of the code per se, but instead improve the robustness or usability of the program e.g. adding a try catch statement. This does not include changing the syntax of print statements or adding comments to the program.
	First change on line referred to in error message	Where a student makes changes to the line that the error message is pointing to immediately after running the program for the first time.
3	**Introduced errors**	Any change that introduces an error that was not existent in the program before the change was made.
	Reverted corrective change(s)	The undoing of a previous corrective change made by the student, in turn introducing an error that they had successfully resolved.
	Introduced additional syntax errors	Changes that add a syntax error to the program's state, in addition to those already in the program.
	Introduced syntactical changes involving **existing statements**	A sub-category encompassing changes made to statements (on entire line of code or more) already existing in the debugging exercises.

	Incorrect syntactical changes involving **output statements**	Edits made to the syntax of an output statement (print in Python) that introduce an additional syntax error.
	Incorrect syntactical changes involving **selection**	Edits made to any part of a selection statement (usually on the line containing the condition) which add an additional syntax error.
	Incorrect syntactical changes involving **iteration**	Edits made to any part of a loop's syntax (always while loops in the debugging exercises) that add an additional syntax error.
	Incorrect syntactical changes to **variable assignments**	Edits made to a line containing a variable assignment that add an additional syntax error.
	Incorrect syntactical changes involving **other statements in program**	Changes made to the over-arching syntax of statements not included in the above categories that introduce an additional syntax error.
	Incorrect syntactical changes involving **symbols**	Changes that involve the addition, removal, or editing of symbols in the debugging exercises that add an additional error. A symbol is defined a token or string of significance in Python e.g. =, *, "".
	Incorrect syntactical changes to **operators**	Where the syntax of a mathematical or logical operator is made inside the program.
	Mathematical operators	Changes involving operators that are used for arithmetic in Python (+, -, *, /, =).
	Logical operators	Changes involved in deciding logic in Python (e.g. ==, and, or) that add an additional syntax error.
	Incorrect syntactical changes to **other symbols**	Any other changes to symbols not mentioned in the above categories that introduce an error e.g. brackets.
	Incorrect syntactical changes involving fusion or distribution of **multiple lines of code**	Changes involving two or more lines of code being being merged, incorrectly switched around , or one line of code being split apart.

	Addition of **non-Python strings**	Examples of students adding strings that do not exist in the grammar of Python to a program e.g. plain English words or syntax from another programming language.
	Incorrect syntactical changes involving **variable references**	Any change involving a reference to a particular variable that adds a syntax error e.g. entirely removing a variable reference.
	Incorrect syntactical changes to **whitespace of program**	Changes to the program's whitespace that introduce an additional syntax error e.g. indenting a single line of code not succeeding a control structure. Bear in mind that change in whitespace may also add logical or runtime errors.
	Introduced runtime errors	Changes that add a runtime error to the program's state, in addition to those already in the program.
	Incorrect semantic changes to **variable assignments**	Changes involving variable assignments that introduce an additional runtime error to the program e.g. replacing an = with == in an assignment (this causes a NameError, an instance of a runtime error).
	Incorrect semantic changes to **variable references**	Changes involving references to a particular variable that add an additional runtime error in the program e.g. adding a variable reference that is not used within the program.
	Introduced type errors	Changes that add a type error to the program's state, in addition to those already in the program.
	Incorrect typewise changes involving **variables**	Incorrect type-changes that involve variables in some manner, be it references or assignments e.g. not casting a variable to an int before performing some logic on it.
	Incorrectly typewise changes to **function call**	Changes involving calls to a function that introduce an additional type error, including incorrect changing the type of parameters.
	Introduced logical errors	Changes that add a logical error to the program's state, in addition to those already in the program.

	Logically incorrect changes involving **output statements**	Any change to the syntax of an output statement, or the text that it outputs that introduces an additional logical error e.g. printing out the name of a variable rather than a reference to the value it is storing.
	Logically incorrect changes to **operators**	Logically incorrect changes to logical or mathematic tokens in the debugging exercises.
	Mathematical operators	Changes to mathematical operators (includes +, -, =, inequalities) that introduce an additional logical e.g. incorrectly changing an inequality that should be >= to ==
	Logical operators	Changes to logical operators that negatively impact the program's correctness e.g. incorrectly replacing an or with an and.
	Logically incorrect changes to **variable assignments**	Edits made relating to variable assignments (that change the semantics of the statements) that introduce an additional logical error e.g. reassigning a variable to have a logically incorrect value.
	Logically incorrect changes involving **variable references**	Edits made that include a particular variable reference that introduce an additional logical error.
	Logically incorrect changes to **program flow**	Changes that incorrectly change the program flow to be logically incorrect. Again, this may be in addition to parts of the program flow that are already incorrect.
	Logically incorrect changes involving **other statements**	Changes focused on any other single statement not mentioned in this sub-category that add a logical statement.
4	**Resolved errors**	Any change that removes one or more of the errors originally included in any of the debugging exercises
	Resolved syntax error	Changes that resolve a single syntax error inside the program.
	Correctly resolved syntax error	The resolution of a syntax error in a manner that removes it completely, such that the part of the program containing the error is completely correct. This code resembles the change that provides the best (and often easiest) fix.

	Resolved error by changing syntax of erroneous component	Resolving a syntax error by refactoring the statement to have different syntax. This often manifests in fixing an erroneous print statement by changing its syntax to printf syntax.
	Introduced logical error from resolution of syntax error	A "shortcut" change where a student removes a syntax error but adds a logical error in the process. For example, a student might add a missing quote in the wrong location, resolving the syntax error but leading to the printing of incorrect output.
	Hard-coded resolution of syntax error	Changes that resolve the syntax error in question by providing a fix that only pertains to the specific error rather than any other similar one, such that adding functionality to the program may still lead to some error in some cases.
	Resolved runtime error	Changes that resolve a single runtime error inside the program.
	Correctly resolved runtime error	The resolution of a runtime error in a manner that removes it completely, such that the part of the program containing the error is completely correct. This code resembles the change that provides the best (and often easiest) fix.
	Introduced logical error from resolution of runtime error	Resolving a runtime error in a logical incorrect manner, leading to the runtime error being replaced by a logical error in the same location in the program.
	Resolved type error	Any change that removes a type error (indicated by an error message beginning with TypeError) existing within the program.
	Correctly resolved type error	The resolution of a type error in a manner that removes it completely, such that the part of the program containing the error is completely correct. This code resembles the change that provides the best (and often easiest) fix.

	Hard-coded resolution of type error	Changes that resolve the type error in the context of this debugging exercises but may lead to the program being incorrect if it were to be expanded upon.
	Resolved logical error	Changes that resolve a single logical error inside the program.
	Correctly resolved logical error	The resolution of a logical error in a manner that removes it completely, such that the part of the program containing the error is completely correct. This code resembles the change that provides the best (and often easiest) fix.
	While code not running	Correction of logical error in runs where the program still contains errors that prevent the program from running.
	After a single successful execution	Logical corrections that occur after a single successful execution of the program.
	After repeated successful executions	Logical corrections that occur after more than one successful executions of the program. Should also be coded in conjunction with “repeated ran **successfully executing** program”.
	Hard-coded resolution of logical error	Changes that resolve the logical error in the context of this debugging exercises but may lead to the program being incorrect if it were to be expanded upon e.g. manually printing output that should be calculated within the program.
5	**Inconsequential changes**	Changes that have no effect on the state of the program, neither resolving nor introducing errors.
	Changes involving **existing statements** in the program	Changes to statements, constructs, or components already present within the program that do not affect its correctness.
	Outputs	Any edits made involving the program output, be it either the syntax of the output statement or the text that is outputted.
	Inputs	Any edits made involving the program input, be it either the syntax of the statement or the text prompting an input.

	Reserved words	Changes relating to keywords in Python which do not add or remove any errors e.g. while, if, for.
	Other statements	Any changes to statements, constructs, or components of the program not included in the above sub-category components.
	Addition, removal or editing of **comments**	Changes revolving around comments in the program, which have no ability to affect the correctness of the program.
	Changes to **symbols**	Changes that involve the addition, removal, or editing of symbols in the debugging exercises that do not add or remove an error from the program. A symbol is defined a token or string of significance in Python e.g. =, *, ""
	Changes to **variable references**	Inconsequential changes made that involve one or more references to a single variable e.g. changing the name of a variable in a semantically correct manner.
	Changes to **whitespace of program**	Changes involving whitespace with no effect on program state e.g. adding or removing a blank line.
	Addition of **unnecessary code**	Constructs that do not do anything in contributing towards correctness of code
6	Miscellaneous behaviour	Changes that do not seem to be directed in any manner towards solving a particular error in the debugging exercises
	Repeated runs	Categories related to the repeated running of incorrect or correct code with no changes in between.
	Repeated ran **successfully executing** program	Two or more consecutive runs of a successfully executing program (doesn't throw an error message), excluding when this behaviour happens at the start of the exercise.

	Repeatedly ran **erroneous** program	Two or more consecutive runs of an erroneous program (which throws an error message), excluding when this behaviour happens at the start of the exercise.
	Repeatedly ran program at **beginning** of exercise	Initially running the code more than once before making any changes at the beginning of the exercise.
	Reverted previous changes	Where part of the program is restored to a previous state, or where a change from a consecutively previous run was undone.